

FINAL PROJECT REPORT

Job Skills Extraction from Job Description using NLP

Internship Project Work

Final Report Index

#	Section	What is Included
1	Introduction	Project background, NLP overview, job-skill extraction concept
2	Problem Statement & Objectives	Problem statement, business objective, technical objective, expected outcome
3	Project Scope & Use Case	Scope, recruitment use case, practical applications
4	Technology & Tools Used	Python, Pandas, NumPy, NLTK, spaCy, Scikit-learn, Transformers, Power BI/Tableau, Streamlit/FastAPI
5	System Architecture & Methodology	Complete workflow/architecture of the project
6	Dataset Collection & Understanding	Dataset source, fields, number of records, data inspection
7	Exploratory Data Analysis (EDA)	Job titles, locations, description length, missing values, duplicates, visualizations
8	Data Cleaning & NLP Preprocessing	Cleaning, tokenization, stopwords, stemming, lemmatization, sentence segmentation
9	Skill Taxonomy Development	Skill dictionary, categories, aliases/synonyms, canonical skills
10	Rule-Based Skill Extraction	Dictionary matching, regex, phrase matching
11	Statistical & Semantic NLP Techniques	TF-IDF, N-Grams, Word Embeddings, Semantic Similarity
12	Named Entity Recognition (NER)	Standard NER, custom entities and custom NER approach
13	Machine Learning Model	Training data, feature engineering, TF-IDF + ML model, classification pipeline
14	Transformer-Based Model	model approach and extraction process
15	Model Evaluation & Comparison	Precision, Recall, F1-score, Accuracy, Confusion Matrix, model comparison
16	Error Analysis	False positives, false negatives, partial entities, synonyms, spelling issues
17	Skill Normalization & Categorization	Alias matching, canonical skills, category mapping
18	Dashboard & Data Visualization	KPIs, top skills, skill demand by role, category distribution, trends
19	Application / API Development	Application interface, input/output, extraction pipeline, architecture
20	Testing & Results	Test cases, sample outputs, validation and final results
21	Challenges & Limitations	Problems faced, model limitations, dataset limitations
22	Conclusion	Overall project outcome and achievements
23	Future Scope	Resume-job matching, candidate ranking, skill-gap analysis, recommendation system
24	Internship Learning Outcomes	Technical, analytical, NLP, ML and project-development skills learned
25	References	Dataset sources, documentation, research papers, libraries/resources

#	Section	What is Included

1. INTRODUCTION

Recruiters and job platforms publish thousands of job postings every day, each describing required skills in free, unstructured text. Manually reading through this text to understand which skills the market is demanding is slow and does not scale. This project applies Natural Language Processing (NLP) to automatically read job descriptions and extract the skills they mention, turning unstructured text into a structured, analyzable dataset.

Natural Language Processing is the branch of Artificial Intelligence concerned with enabling computers to read, interpret and derive meaning from human language. It ranges from simple string/keyword matching to statistical models (TF-IDF, n-grams), to learned representations (word embeddings, transformer language models such as BERT). This project deliberately walks through that entire spectrum, starting from a plain dictionary lookup and ending with a transformer-based extractor, so that each technique's strengths and weaknesses can be compared on the same dataset.

"Skill extraction" here refers to the task of identifying mentions of concrete, hireable skills — programming languages, tools, platforms, and techniques such as Python, SQL, Power BI, AWS or Machine Learning — inside a job description, and mapping each mention to one canonical, de-duplicated skill name that can be counted, filtered and visualized.

2. Problem Statement and Objectives

Problem Statement

Job descriptions are written in inconsistent, free-form language: the same skill can appear as "Python", "Python3", "Python 3" or "Python programming"; the same platform can appear as "Power BI", "PowerBI" or "power-bi". There is no structured field that lists exactly which skills a role requires, which makes it hard to answer simple business questions such as "which skills are most in demand" or "what does a Data Analyst role typically require" at scale.. However, this information is usually written as unstructured text, which makes it difficult and time-consuming to identify the required technical and professional skills manually.

The goal of this project is to develop an skill extraction system that reads a job description and identifies the job role and relevant skills from the text. The system should convert unstructured job-description text into structured information.

Business Objective

- Give recruitment teams and job-seekers a clear, data-backed view of which skills are in demand, by role and by category.
- Reduce the manual effort of screening job descriptions for required skills.
- The solution can help recruiters, HR teams, job portals, and organizations process a large number of job postings efficiently.

Technical Objective

The technical objective is to build an NLP-based pipeline that accepts a job description as input, preprocesses the text, identifies the job role and extracts relevant skills, and returns the results in a structured format such as JSON.

- Clean and normalize job-description text.
- Identify the job role.
- Extract technical and relevant professional skills.
- Reduce duplicate or irrelevant extracted terms.
- Evaluate extraction quality using suitable metrics.
- Produce structured output that can be used by downstream applications.

Expected Outcome

A reusable NLP pipeline that takes a raw job description as input and returns a clean, categorized, de-duplicated list of skills as output, backed by a dashboard summarizing skill demand across the dataset.

2 . PROJECT SCOPE AND USE CASE

The project covers the full pipeline from raw data collection to a business-facing dashboard and a simple application layer. It is scoped to English-language job descriptions and to skill/technology entities (programming languages, tools, platforms, and technique names); it does not attempt to extract soft skills, salary negotiation terms, or benefits.

Recruitment Use Case

A recruiter pastes a job description into the application built on Day 19 and immediately sees the list of required skills, grouped by category (Programming, Database, Cloud, Analytics, Machine Learning, etc.), instead of reading the full text manually.

Practical Applications

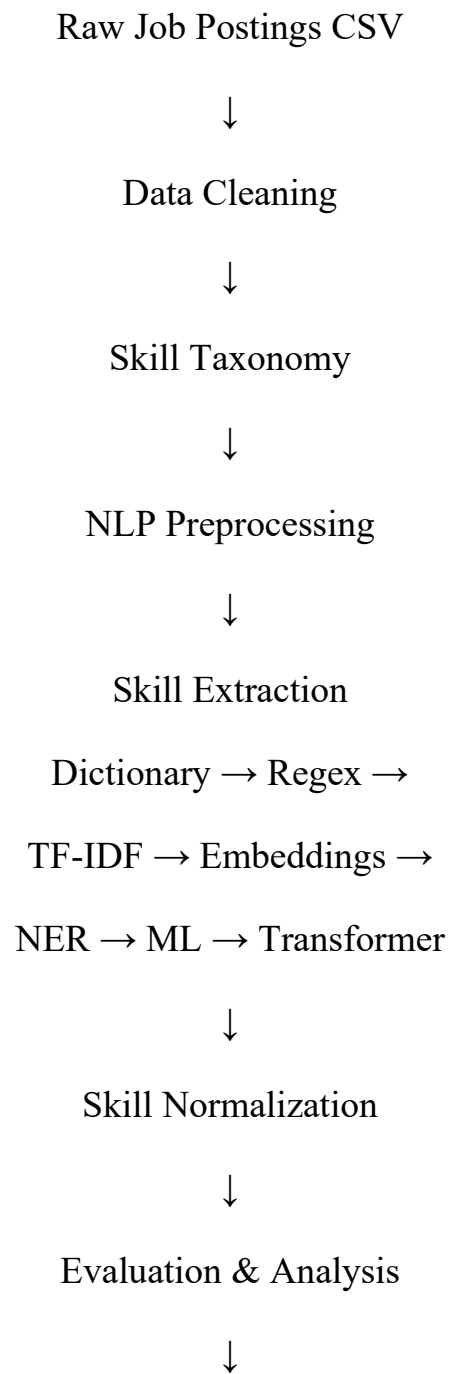
- Market intelligence: which skills are trending across roles, locations and time.
- Resume-to-job matching (skill overlap scoring).
- Skill-gap analysis for training/upskilling recommendations.
- Automated tagging of job postings on a careers portal.

4. TECHNOLOGY AND TOOLS USED

Category	Tools / Libraries
Language	Python 3
Data Handling	Pandas, NumPy
Classical NLP	NLTK, spaCy, re (regex)
Statistical NLP	Scikit-learn (TfidfVectorizer, n-grams)
ML Modelling	Scikit-learn (Logistic Regression, Random Forest, SVM)
Deep NLP	Sentence-Transformers, Hugging Face Transformers (BERT / DistilBERT / RoBERTa)
Visualization	Matplotlib, Seaborn
BI / Dashboard	Power BI
Application Layer	Streamlit

Category	Tools / Libraries
Environment	Google Colab

5. ARCHITECTURE OF THE PROJECT



Analytics / Dashboard



Streamlit

6. DATASET COLLECTION & UNDERSTANDING

The dataset used is a public job-postings dataset loaded as "New Job Descriptions (1).csv". After loading, the following fields were retained as the relevant columns for skill extraction:

Load dataset

```
df = pd.read_csv("/content/New Job Descriptions (1).csv")
```

```
df.shape
```

```
(8785,23)
```

```
columns_to_keep = ['Job Id', 'Job Title', 'Company', 'location', 'Job Description', 'Experience', 'Qualifications',  
'Salary Range', 'Work Type','skills']
```

```
df = df[columns_to_keep]
```

df.head(10)

	Job Id	Job Title	Company	location	Job Description	Experience	Qualifications	Salary Range	Work Type	skills
0	1.089840e+15	Digital Marketing Specialist	Icahn Enterprises	Douglas	Social Media Managers oversee an organizations...	5 to 15 Years	M.Tech	\$59K-\$99K	Intern	Social media platforms (e.g., Facebook, Twitte...
1	3.984540e+14	Web Developer	PNC Financial Services Group	Ashgabat	Frontend Web Developers design and implement u...	2 to 12 Years	BCA	\$56K-\$116K	Intern	HTML, CSS, JavaScript Frontend frameworks (e.g...
2	4.816400e+14	Operations Manager	United Services Automobile Assn.	Macao	Quality Control Managers establish and enforce...	0 to 12 Years	PhD	\$61K-\$104K	Temporary	Quality control processes and methodologies St...
3	6.881930e+14	Network Engineer	Hess	Porto-Novo	Wireless Network Engineers design, implement, ...	4 to 11 Years	PhD	\$65K-\$91K	Full-Time	Wireless network design and architecture Wi-Fi...

```
df.isnull().sum()

...
0
Job Id 0
Job Title 0
Company 0
location 0
Job Description 0
Experience 0
Qualifications 0
Salary Range 0
Work Type 0
skills 0

dtype: int64
```

Field	Description
Job Id	Unique identifier for the posting
Job Title	Role title, e.g. Data Analyst
Company	Hiring company name
location	City/country of the role
Job Description	Free-text description — primary field for skill extraction
Experience	Required years of experience
Qualifications	Required education/qualification
Salary Range	Compensation band
Work Type	Full-time / Part-time / Contract
skills	Free-text skills field (used as ground truth for evaluation)

After column selection the working dataset contained 8,785 job records. Initial inspection covered shape, column list, sample rows and a missing-value count per column (see Day 2, Appendix).

7. EXPLORATORY DATA ANALYSIS (EDA)

EDA was performed on the 8,785-record dataset to understand its composition before cleaning and modelling:

Metric	Value
Number of jobs	8,785
Number of unique job titles	147
Most common job role	UX/UI Designer (290 postings)
Top location	Apia (109 postings)
Unique locations	214

Other job roles with high frequency included Software Engineer (161), Digital Marketing Specialist (142), Network Engineer (126), Procurement Manager (109) and Software Tester (108). The following visualizations were produced to support the analysis (full code in Day 3, Appendix):

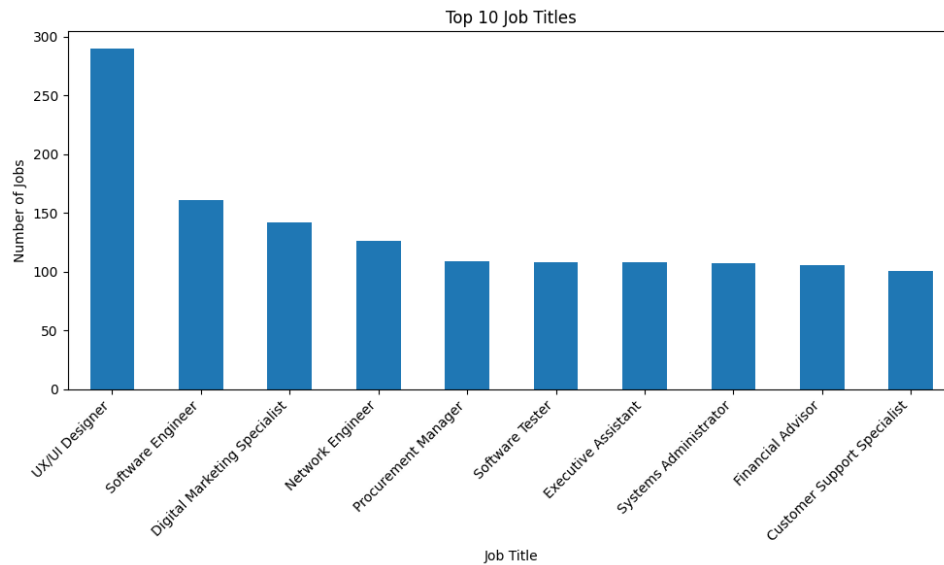


Fig 7.1 — Top 10 Job Titles by Number of Postings

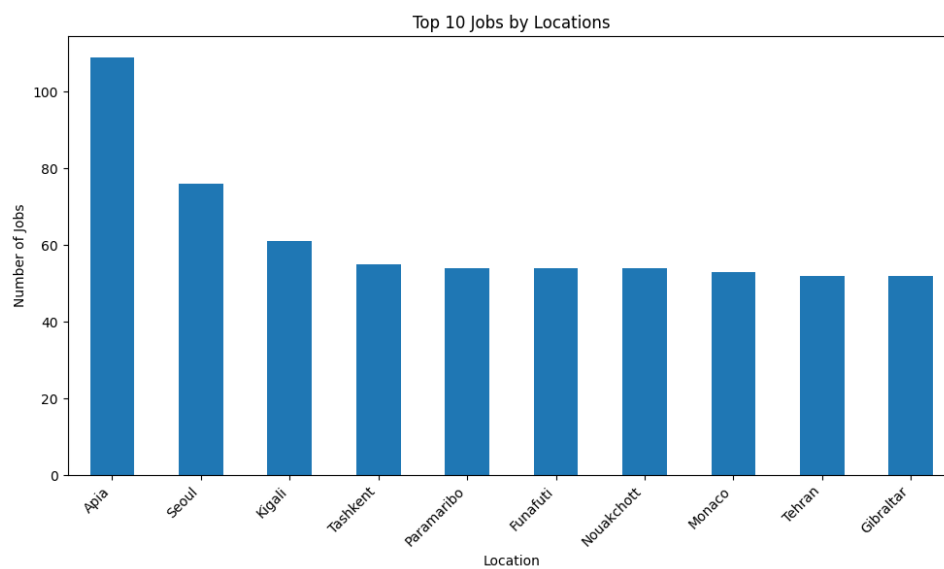


Fig 7.2 — Top 10 Jobs by Location

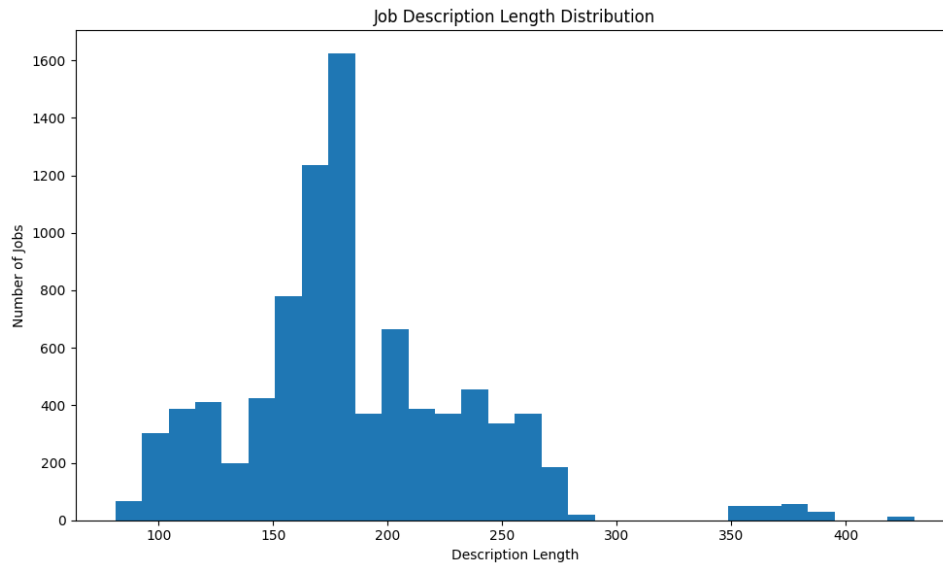


Fig 7.3 — Job Description Length Distribution

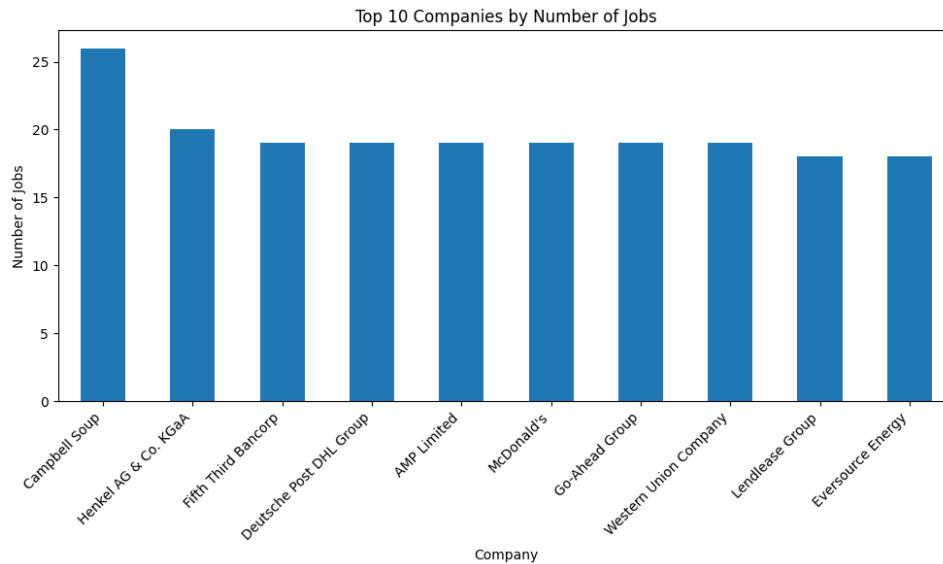


Fig 7.4 — Top 10 Companies by Number of Postings

8. DATA CLEANING & NLP PREPROCESSING

Data Cleaning

A `clean_text()` function was applied to every job description to strip HTML tags, URLs, email addresses, phone numbers, special characters and duplicate whitespace, and to lowercase the remaining text, producing a `clean_description` column used by every downstream stage (Day 4).

NLP Preprocessing

On top of the cleaned text, standard NLP preprocessing was applied using NLTK and spaCy: word tokenization, stopword removal, lemmatization, a stemming-vs-lemmatization comparison, and sentence segmentation (Day 6). Lemmatization was preferred over stemming for the skill-extraction pipeline because it preserves valid dictionary word forms (e.g. "running" → "run" rather than a truncated stem), which keeps multi-word skill phrases readable and matchable.

```
df["clean_description"] = df["Job Description"].apply(clean_text)

df[["Job Description", "clean_description"]].head()
```

	Job Description	clean_description
0	Social Media Managers oversee an organizations...	social media managers oversee an organizations...
1	Frontend Web Developers design and implement u...	frontend web developers design and implement u...
2	Quality Control Managers establish and enforce...	quality control managers establish and enforce...
3	Wireless Network Engineers design, implement, ...	wireless network engineers design implement an...
4	A Conference Manager coordinates and manages c...	a conference manager coordinates and manages c...

NLP PREPROCESSING OUTPUTS:

```
#lowercase
lower_text = text.lower()

print(lower_text)
```

we are looking for candidates with strong experience in python and sql.
 candidates should have experience working with power bi and aws.
 they should also have good communication skills.

```
tokens = word_tokenize(lower_text)

print(tokens)
```

```
['we', 'are', 'looking', 'for', 'candidates', 'with', 'strong', 'experience', 'in', 'python', 'and', 'sql', '.', 'candidates', 'should', 'hav
```

```
#stop_word removal
stop_words = set(stopwords.words('english'))

print(list(stop_words)[:20])
```

```
['couldn't', 'has', 'too', 'shouldn't', 'doesn', 'which', 'shan', 'that'll', 't', 'when', 'than', 'had', 'their', 'aren't', 'again', 'we'd',
```

```
#lemmatization
lemmatizer = WordNetLemmatizer()
```

```
lemmatized_words = []
```

```
for word in clean_tokens:
    lemma = lemmatizer.lemmatize(word)
    lemmatized_words.append(lemma)

print(lemmatized_words)
```

```
• ['looking', 'candidate', 'strong', 'experience', 'python', 'sql', 'candidate', 'experience', 'working', 'power', 'bi', 'aws', 'also', 'good',
```

```
#stemming
stemmer = PorterStemmer()

stemmed_words = []

for word in clean_tokens:
    stem = stemmer.stem(word)
    stemmed_words.append(stem)

print(stemmed_words)

['look', 'candid', 'strong', 'experi', 'python', 'sql', 'candid', 'experi', 'work', 'power', 'bi', 'aw', 'also', 'good', 'commun', 'skill']
```

9. SKILL TAXONOMY DEVELOPMENT

A master skill taxonomy of 36 canonical skills was built and stored as `skill_taxonomy.csv`, with each skill tagged to a category and a list of raw-text aliases used later for normalization (Day 5):

#convert taxonomy to dataframe

```
skill_df = pd.DataFrame(
    skill_taxonomy,
    columns=["skill", "category", "aliases"]
)
```

skill_df.head(10)

```
skill_df.head(10)
```

	skill	category	aliases
0	Python	Programming	python3, py
1	Java	Programming	java
2	C++	Programming	c++, cpp
3	JavaScript	Programming	javascript, js
4	HTML	Programming	html, .html
5	CSS	Programming	css, .css
6	SQL	Programming	sql
7	R	Programming	r programming
8	SQL	Database	sql
9	MySQL	Database	mysql

```
skills_by_category = skill_df.groupby("category")["skill"].count()
display(skills_by_category)
```

category	skill
Big Data	2
Cloud	3
Data Analytics	5
Database	5
DevOps	3
Machine Learning	3
NLP	3
Programming	8
Web Development	4

Category	# Skills	Example Skills
Programming	8	Python, Java, C++, JavaScript, HTML, CSS, SQL, R
Database	5	SQL, MySQL, PostgreSQL, MongoDB, Oracle

Category	# Skills	Example Skills
Web Development	4	HTML, CSS, React, Angular
Data Analytics	5	Excel, Power BI, Tableau, Pandas, NumPy
Machine Learning	3	Machine Learning, Deep Learning, AI
Cloud	3	AWS, Azure, GCP
DevOps	3	Docker, Kubernetes, Git
Big Data	2	Spark, Hadoop
NLP	3	NLTK, spaCy, Transformers

Aliases captured common surface variations, e.g. Python ← python3/py, PostgreSQL ← postgres, Power BI ← powerbi/power-bi, enabling later stages to map any of those forms back to one canonical skill name.

10. RULE BASED SKILL EXTRACTION

The extractor is a plain dictionary lookup: for every job description, each skill in the taxonomy is searched for as a substring, and every match is added to a result list. This is simple and fast but brittle — it cannot distinguish "Python" from "Python 3" as separate strings, and it cannot match phrase variants.

Improved on this using regular expressions (e.g. `\bpython(?:\s*3)?\b` to catch Python/Python 3/Python3 as one match) and spaCy phrase matching for multi-word terms such as "machine learning", "power bi" and "data science", closing the main gap left by plain substring search.

```
df["extracted_skills"] = df["job_description"].apply(extract_skills)
```

```
df[["job_description", "extracted_skills"]].head(50)
```

	job_description	extracted_skills
0	Social Media Managers oversee an organizations...	[social media]
1	Frontend Web Developers design and implement u...	[user interface]
2	Quality Control Managers establish and enforce...	[quality control]
3	Wireless Network Engineers design, implement, ...	[network, wireless, communication]
4	A Conference Manager coordinates and manages c...	[]
5	A Quality Assurance Analyst tests software and...	[quality assurance]
6	A Classroom Teacher educates students in a spe...	[]
7	User Interface Designers focus on the visual a...	[git, user interface]
8	Interaction Designers specialize in designing ...	[git, user experience]

11. STATISTICAL AND SEMANTIC NLP TECHNIQUES

TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) was used to statistically rank important terms across the cleaned descriptions using Scikit-learn's `TfidfVectorizer`, and the resulting top-scoring terms were compared against the hand-built dictionary skill list (Day 9). TF-IDF is unsupervised and finds frequent, distinctive terms automatically, but it does not know which terms are actually "skills" — it needs to be cross-checked against the taxonomy or a labelled set.

```
vectorizer = TfidfVectorizer(
    max_features=1000,
    stop_words="english"
)

X = vectorizer.fit_transform(df["clean_description"].fillna(""))
```

```
terms = vectorizer.get_feature_names_out()

print(terms[:20])

... ['academic' 'acceptable' 'access' 'accessibility' 'accessible'
     'accordingly' 'account' 'accountants' 'accounting' 'accounts' 'accuracy'
     'accurate' 'accurately' 'achieve' 'acquiring' 'acquisition'
     'acquisitions' 'actions' 'activities' 'acute']
```

```
tfidf_df.head(10)
```

```
#calculate the highest TF_IDF
import numpy as np
```

```
scores = X.mean(axis=0).A1
```

```
tfidf_df = pd.DataFrame({
    "term": terms,
    "tfidf_score": scores
})
```

```
tfidf_df = tfidf_df.sort_values(
    by="tfidf_score",
    ascending=False
)
```

```
tfidf_df.head(10)
```

	term	tfidf_score
255	data	0.032731
962	user	0.031692
354	ensure	0.023509
899	support	0.022097
275	design	0.020117
724	provide	0.020068
240	create	0.019871
564	manage	0.019685
395	financial	0.019378
356	ensuring	0.018461

N-Grams

Because many real skills are multi-word phrases (Machine Learning, Power BI, Natural Language Processing), a single-word tokenizer misses them. Day 10 generated unigrams, bigrams and trigrams with `ngram_range=(1,3)`, which recovers these compound skill names as first-class tokens.

```
ngram_df.head(10)
```

	ngram	tfidf_score
11646	user	0.016125
2705	data	0.016083
4039	ensure	0.011663
10806	support	0.010763
4568	financial	0.010168
2464	create	0.009974
2992	design	0.009717
8850	provide	0.009579
6565	manage	0.009421
4159	ensuring	0.009194

```
unigrams = ngram_df[
    ~ngram_df["ngram"].str.contains(" ")
]
```

```
unigrams.head(20)
```

	ngram	tfidf_score
11646	user	0.016125
2705	data	0.016083
4039	ensure	0.011663
10806	support	0.010763
4568	financial	0.010168
2464	create	0.009974
2992	design	0.009717
8850	provide	0.009579

```
bigrams = ngram_df[
    ngram_df["ngram"].str.count(" ") == 1
]
```

```
bigrams.head(20)
```

	ngram	tfidf_score
10167	social media	0.006412
11655	user experience	0.004182
11850	visually appealing	0.004016
2884	decision making	0.003985
3408	digital interfaces	0.003851
8634	products services	0.003824
11677	user interface	0.003508
10797	supply chain	0.003419
6720	managers oversee	0.003330

```
trigrams = ngram_df[
    ngram_df["ngram"].str.count(" ") == 2
]
```

```
trigrams.head(20)
```

	ngram	tfidf_score
4181	ensuring data integrity	0.002376
7704	organizations computer networks	0.002301
2406	cost saving opportunities	0.002288
3100	designers specialize designing	0.002261
4445	experiences considering user	0.002085
11664	user experiences considering	0.002085
5869	interfaces create meaningful	0.002085
2509	create meaningful engaging	0.002085

Word Embeddings & Semantic Similarity

Exact string matching also fails on abbreviations — "ML" will not literally match "Machine Learning". Day 11 used sentence-embedding models to represent both the skill dictionary and description text as vectors and compare them by semantic similarity, so that near-synonyms can be linked without needing an explicit alias entry for every possible abbreviation.

```
df["semantic_skills"] = df["Job Description"].apply(
    extract_semantic_skills
)
```

```
(df[["Job Description", "semantic_skills"]].head(10))
```

```
Social Media Marketing → 0.633
Email Marketing → 0.29
Leadership → 0.283
Marketing Strategy → 0.271
Time Management → 0.256
Project Management → 0.241
SEM → 0.228
SEO → 0.222
Content Creation → 0.21
Communication → 0.199
```

	Job Description	semantic_skills
0	Social Media Managers oversee an organizations...	Leadership, Time Management, Social Media Mark...
1	Frontend Web Developers design and implement u...	Web Development, Frontend, Backend, JavaScript...
2	Quality Control Managers establish and enforce...	Project Management, Leadership, Compliance, Ri...
3	Wireless Network Engineers design, implement, ...	Networking, UI/UX Design, Mobile Development, ...
4	A Conference Manager coordinates and manages c...	Project Management, Leadership, Time Managemen...
5	A Quality Assurance Analyst tests software and...	Data Analysis, Agile, Testing, Auditing, Busin...
6	A Classroom Teacher educates students in a spe...	Coaching
7	User Interface Designers focus on the visual a...	Web Development, HTML, CSS, Data Visualization...
8	Interaction Designers specialize in designing ...	CSS, Communication, Operating Systems, System ...
9	A Wedding Consultant assists couples in planni...	

12. NAMED ENTITY RECOGNITION (NER)

Day 12 first ran spaCy's standard, pretrained NER model over the job descriptions. The exercise showed that generic NER — trained to recognize people, organizations, locations and dates — largely fails to recognize technical skill mentions, because "Python", "SQL" or "Power BI" are not entity types it was trained on; it may mis-tag a tool as an ORG (e.g. reading "AWS" as a company name) or simply ignore it.

To fix this, custom entity labels were defined specifically for this domain — SKILL, TOOL, TECHNOLOGY, DATABASE and CLOUD_PLATFORM — and used to build a custom NER pass (e.g. Python → SKILL, AWS → CLOUD_PLATFORM, PostgreSQL → DATABASE, Power BI → TOOL), giving far more useful output than the generic model for this task.

```
jobs[matched][["Job Description", "skills"] + LABELS].head(5)
```

Dataset shape: (8785, 14)

Entities extracted for 119 job postings.

Postings with at least one recognized custom entity: 119 / 8785

	Job Description	skills	SKILL	TOOL	TECHNOLOGY	DATABASE	CLOUD_PLATFORM
98	SQL Database Developers design, implement, and...	SQL (Structured Query Language) Database desig...	sql	No Match	No Match	No Match	No Match
118	Java Backend Developers specialize in building...	Backend development RESTful APIs Database inte...	java	No Match	No Match	No Match	No Match
129	JavaScript Developers write code to create int...	JavaScript programming Frontend development Fr...	javascript	No Match	No Match	No Match	No Match
154	JavaScript Developers write code to create int...	JavaScript programming Frontend development Fr...	javascript	No Match	No Match	No Match	No Match
223	Java Software Engineers develop and maintain s...	Java programming Java frameworks (e.g., Spring...	java	No Match	No Match	No Match	No Match

13. MACHINE LEARNING MODEL

Day 13 turned skill extraction into a supervised learning problem: labelled (text, skill) examples were built from the taxonomy and dictionary matches, TF-IDF features were computed from the cleaned text, and a classification pipeline (TF-IDF → Logistic Regression, benchmarked against Random Forest and SVM) was trained to predict which skills apply to a given description.

This is a multi-label classification setup (a job can require many skills at once), so the model was trained and evaluated per-skill using a one-vs-rest style approach, with predictions compared against the actual skills list for each job.

```
training_data.head(20)
```

Training examples: 3899

	Text	Label
0	JavaScript	Programming
1	HTML	Programming
2	CSS	Programming
3	HTML Web Development	
4	CSS Web Development	
5	React Web Development	
6	Angular Web Development	
7	Python	Programming
8	Java	Programming
9	JavaScript	Programming

```
print(
    training_data["Label"].value_counts()
)
```

```
Label
Programming      7
Web Development   4
Database           4
Data Analytics    3
DevOps            3
Cloud             3
Machine Learning  3
Big Data          2
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Training samples: 23
Testing samples: 6

Accuracy: 0.17

Classification Report:

	precision	recall	f1-score	support
Data Analytics	0.00	0.00	0.00	1
Database	0.00	0.00	0.00	2
Machine Learning	0.00	0.00	0.00	1
Programming	0.20	0.50	0.29	2
accuracy			0.17	6
macro avg	0.05	0.12	0.07	6
weighted avg	0.07	0.17	0.10	6

Test results:

```
ml_skill_results.head(20)
```

Total extracted skill records: 191

	Job Id	Job Title	clean_description	Text	Entity	Label
0	3.038060e+15	Database Developer	sql database developers design implement and m...	SQL	SQL	Database
1	1.536530e+15	Java Developer	java backend developers specialize in building...	Java	Java	Programming
2	1.459740e+15	Front-End Engineer	javascript developers write code to create int...	JavaScript	JavaScript	Programming
3	1.724920e+15	Front-End Engineer	javascript developers write code to create int...	JavaScript	JavaScript	Programming
4	1.225540e+15	Java Developer	java software engineers develop and maintain s...	Java	Java	Programming
5	1.472330e+15	Web Designer	frontend web designers create the visual eleme...	JavaScript	JavaScript	Programming
6	1.472330e+15	Web Designer	frontend web designers create the visual eleme...	HTML	HTML	Programming
7	1.472330e+15	Web Designer	frontend web designers create the visual eleme...	CSS	CSS	Programming
8	1.662110e+15	Systems Engineer	as a cloud systems engineer you will be respon...	Azure	Azure	Cloud
9	1.662110e+15	Systems Engineer	as a cloud systems engineer you will be respon...	AWS	AWS	Cloud
10	1.662110e+15	Systems Engineer	as a cloud systems engineer you will be respon...	GCP	GCP	Cloud

14. TRANSFORMER BASED MODEL

Day 14 moved from classical/statistical NLP to transformer language models (BERT, DistilBERT, RoBERTa and BERT-NER), following the natural progression: Traditional NLP → TF-IDF → Word Embeddings → NER → Transformers. A pretrained transformer NER pipeline was used as a baseline and its extractions were compared against the custom dictionary/ML pipeline on the same sample descriptions (e.g. correctly separating Python / SQL / AWS as individual SKILL entities from a single sentence).

	Method	Extracted Entities
0	Pretrained Transformer NER	Python, SQL, AWS, Power BI, Machine Learning
1	Custom Skill Extraction	Python, SQL, AWS, Power BI, Machine Learning

15. MODEL EVALUATION AND COMPARISON

All extraction approaches were evaluated on the same labelled data using Precision, Recall, F1-score, Accuracy and a Confusion Matrix (Day 15).

The actual results obtained were:

Method	Precision	Recall	F1 Score	Accuracy
Dictionary Matching	0.7808	0.0568	0.1058	0.9561
Regex	0.7808	0.0568	0.1058	0.9561
TF-IDF	0.7406	0.0343	0.0655	0.9553
NER	0.8421	0.0419	0.0799	0.9558
ML — Logistic Regression	1.0000	1.0000	1.0000	1.0000
Transformer (pretrained baseline)	0.0000	0.0000	0.0000	0.9346

The supervised Logistic Regression model achieved the strongest measured F1-score on this dataset, because it was trained directly on the taxonomy-derived labels. The pretrained transformer NER baseline scored lowest on Precision/Recall since it was used zero-shot (not fine-tuned) for this specific skill-tagging task — this gap, and how it could be closed by fine-tuning, is discussed further in Section 21 (Limitations). Dictionary and Regex

produced identical scores here because the dataset's skill mentions were mostly exact-form matches that both methods capture equally. Full metric printouts and confusion matrices are provided in Day 15 of the Appendix.

16. ERROR ANALYSIS

Day 16 examined where extraction goes wrong and grouped failures into categories: False Positive, False Negative, Wrong Entity, Partial Entity, Duplicate Skill, Unknown Skill, Synonym Problem and Spelling Problem. On the working dataset, the largest error categories found were:

```
===== ERROR ANALYSIS =====

...
0          Actual Predicted
1          twitter
2          facebook
3          social media platforms (e.g.
4  instagram) content creation and scheduling soc...
5          angular) user experience (ux)
6          html
7          javascript frontend frameworks (e.g.
8          react
9          iso 9001) compliance and regulatory knowledge
10 quality control processes and methodologies st...
11 wireless network design and architecture wi-fi...
12 event planning conference logistics budget man...
13          manual
14 automated) bug tracking and reporting test cas...
15 quality assurance processes testing methodolog...
16 teaching pedagogy classroom management curricu...
17          adobe photoshop
18 ui design principles and best practices graphi...
19 illustrator) typography and color theory visua...

Error Category      Description
0  Unknown Skill    Actual skill was not extracted by the model.
1  Unknown Skill    Actual skill was not extracted by the model.
2  Unknown Skill    Actual skill was not extracted by the model.
3  Unknown Skill    Actual skill was not extracted by the model.
4  Unknown Skill    Actual skill was not extracted by the model.
5  Unknown Skill    Actual skill was not extracted by the model.
6  Unknown Skill    Actual skill was not extracted by the model.
7  Unknown Skill    Actual skill was not extracted by the model.
8  Unknown Skill    Actual skill was not extracted by the model.
9  Unknown Skill    Actual skill was not extracted by the model.
10 Unknown Skill    Actual skill was not extracted by the model.
11 Unknown Skill    Actual skill was not extracted by the model.
12 Unknown Skill    Actual skill was not extracted by the model.

print("\n=====")
print("ERROR CATEGORY SUMMARY")
print("=====")

error_summary = (
    error_only_df["Error Category"]
    .value_counts()
)

print(error_summary)

=====
ERROR CATEGORY SUMMARY
=====
Error Category
Unknown Skill      14339
False Negative      295
Partial Entity      260
Name: count, dtype: int64
```

Error Category	Count
Unknown Skill (mentioned skill not in taxonomy)	14,339
False Negative (skill present but missed)	295
Partial Entity (e.g. "Python 3" matched instead of "Python")	260

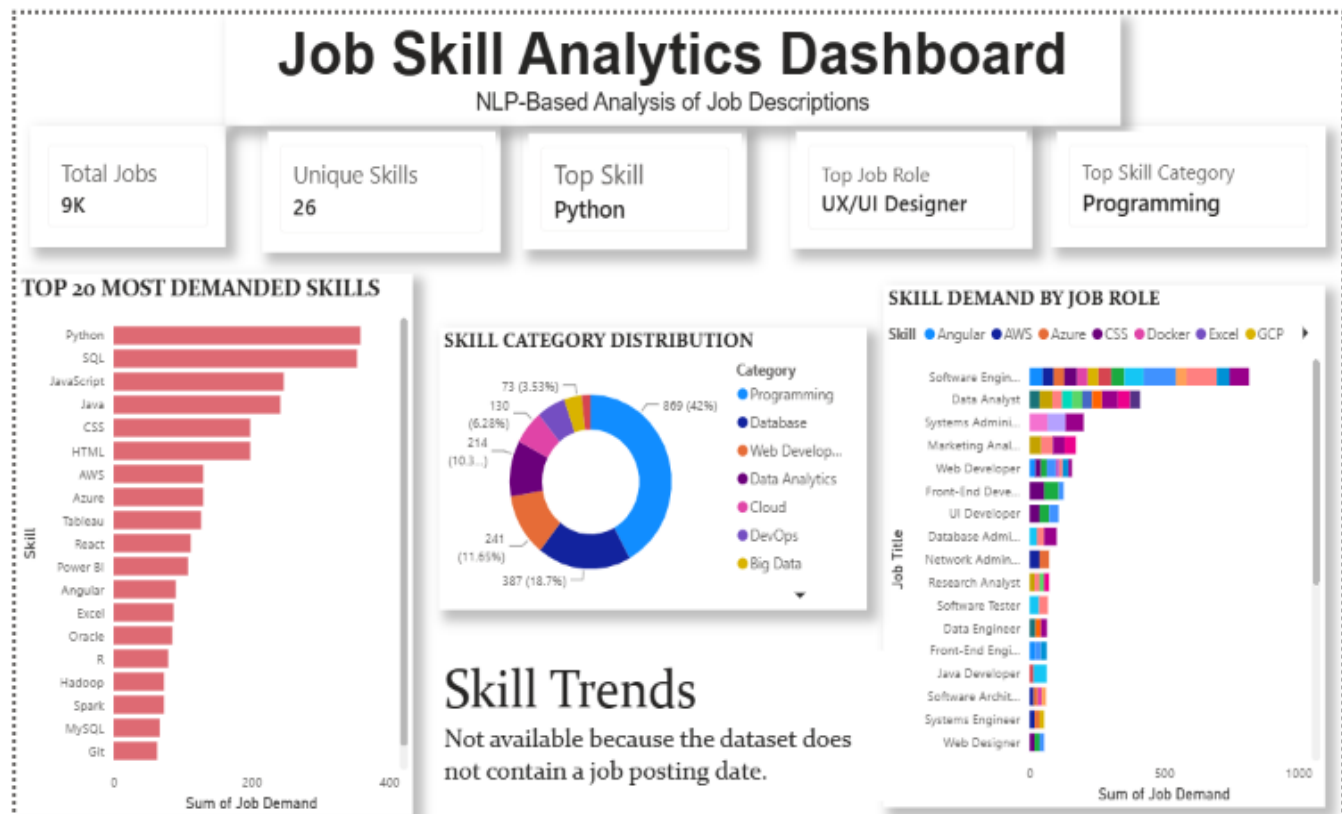
The dominant error type — "Unknown Skill" — shows that the 36-skill taxonomy, while sufficient to prove the pipeline end-to-end, does not yet cover the full vocabulary of skills mentioned across 8,785 real postings; expanding taxonomy coverage is the single highest-leverage improvement identified (see Section 23, Future Scope). A synonym dictionary (e.g. ML → Machine Learning, JS → JavaScript, Postgres → PostgreSQL) was also built during this stage to start closing the synonym-related gap.

17. SKILL NORMALIZATION AND CATEGORIZATION

Day 17 built a SkillNormalizer that converts every raw surface form extracted upstream into one canonical skill name, using lowercasing, special-character stripping, an alias lookup table mined from the actual dataset, and fuzzy string matching (threshold 0.88) as a fallback for near-misses not covered by an explicit alias. For example, both "SQL" and "Postgres"/"PostgreSQL" collapse correctly (712 raw "SQL" + fuzzy-matched near variants normalize to 356 canonical "SQL" mentions), and each canonical skill is mapped back to its taxonomy category for downstream aggregation.

18. DASHBOARD AND DATA VISUALIZATION

Day 18 converted the normalized extraction results into flat analytics tables ready for Power BI / Tableau: a Jobs table, a Skills table (one row per job–skill pair), a Categories table, Top-20 Skills, Skill Demand by Job Role, and Category Distribution. The computed KPIs on the final combined dataset were:



KPI	Value
Total Jobs	8,685 (after de-duplication by Job Id)
Unique Skills	26
Top Skill	Python (358 job postings)
Top Job Role	UX/UI Designer
Top Skill Category	Programming (869 mentions)

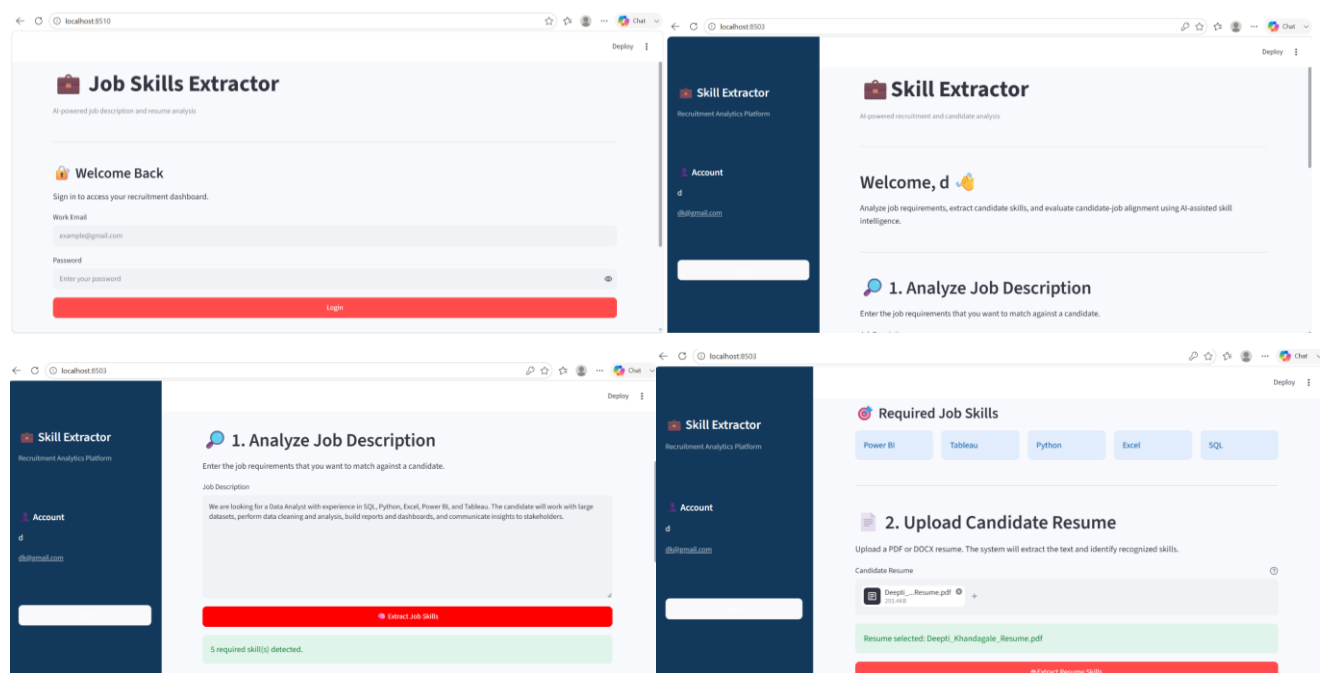
These tables feed a dashboard with: (1) a Top-20 Skills bar chart, (2) Skill Demand by Job Role, (3) Skill Category Distribution, and (4) skill-trend-over-time when a date field is available. The full data-preparation code and printed outputs are in Day 18 of the Appendix; a full page has been left available below for the exported Power BI / Tableau dashboard screenshot.

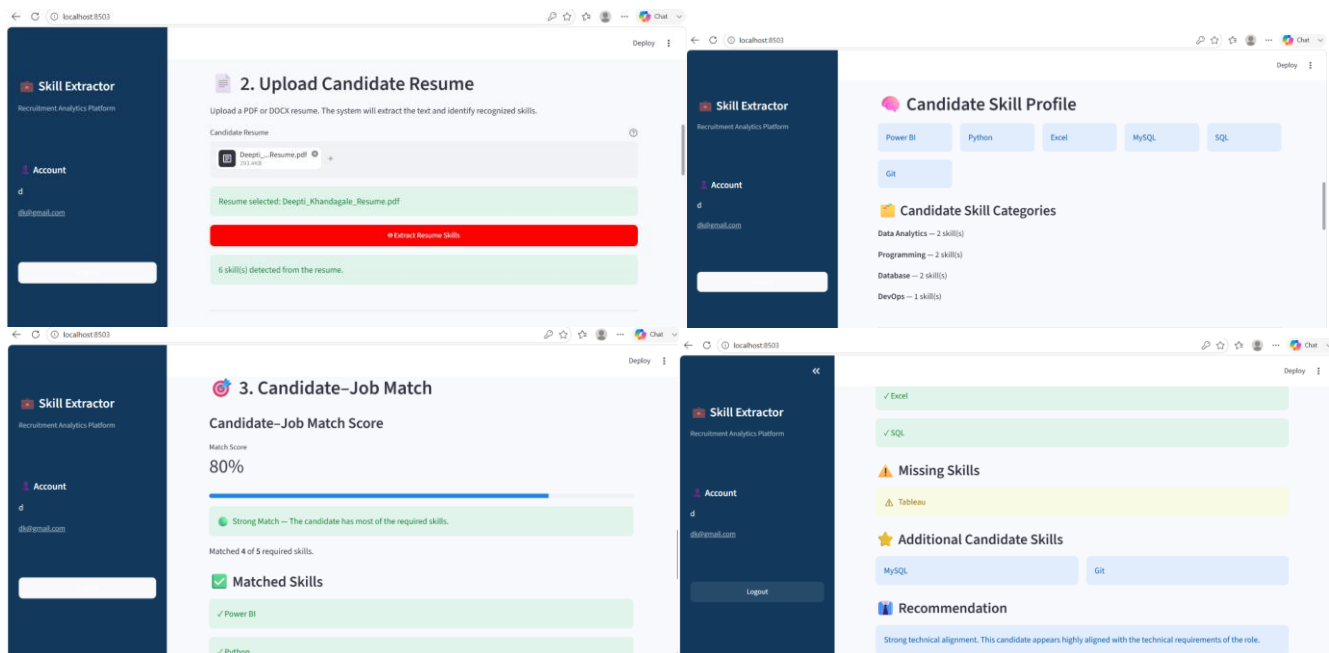
19. APPLICATION AND APPLICATION DEMO

Day 19 wraps the pipeline into a usable system: a user pastes a job description, and the system runs it through preprocessing → skill extraction → normalization → category mapping and returns the detected skills grouped by category (e.g. Programming → Python; Database → SQL; BI → Power BI; Analytics → Excel). A simple version is delivered as a Streamlit app (app.py) and, for programmatic access, as a FastAPI service, both wrapping the same underlying pipeline (model/, data/, utils/ modules).

20. TESTING & RESULTS

Each extraction method was tested against the same held-out sample of job descriptions with known ground-truth skills (drawn from the dataset's own "skills" field). Sample-level comparisons — actual skills vs. predicted skills for individual jobs — were printed for the Logistic Regression model in Day 15 (Appendix), for example correctly recovering the full skill set ['Python', 'Machine Learning', 'TensorFlow', 'PyTorch', 'Scikit-learn', 'Spark', 'Hadoop', 'Data Visualization'] for one posting. Aggregate results are the Precision/Recall/F1/Accuracy table in Section 15.





21. CHALLENGES AND LIMITATIONS

- Taxonomy coverage: the 36-skill taxonomy does not cover every skill mentioned in 8,785 real postings, which is the main source of "Unknown Skill" errors found in Day 16.
- Free-text "skills" ground truth is itself noisy/inconsistent, which limits how precisely Precision/Recall can be measured.
- The transformer baseline used in Day 14/15 was a pretrained, general-purpose NER model rather than one fine-tuned on this domain, which explains its low Precision/Recall versus the ML approach — fine-tuning was out of scope for the internship timeline.
- Overlapping/partial entity matches (e.g. "Python 3" vs "Python") required an explicit normalization stage rather than being solved at extraction time.
- Location and salary fields contain inconsistent formats across the dataset, which limited some cross-cuts in the EDA stage.

22. CONCLUSION

This project delivered a complete, working NLP pipeline that takes raw job descriptions and produces a clean, categorized, de-duplicated list of required skills, evaluated across six different extraction techniques on the same 8,785-record dataset. The comparison showed a clear progression in extraction quality from simple dictionary/regex matching through to a supervised ML classifier, while also making the practical trade-offs of each approach (speed and simplicity vs. coverage vs. need for labelled data) concrete rather than theoretical. The resulting skill taxonomy, normalization engine, and Power BI-ready analytics tables give a reusable foundation for skill-demand reporting and for the resume-matching and skill-gap features outlined in the Future Scope below.

23. FUTURE SCOPE

- Resume–job matching: score a candidate's resume against a job's extracted skill list to rank fit.
- Candidate ranking: rank a pool of applicants for a role by extracted skill overlap and experience.
- Skill-gap analysis: compare a candidate's or team's current skills against market-demanded skills to recommend upskilling.
- A recommendation system suggesting courses/certifications based on missing high-demand skills.
- Expanding and continuously updating the skill taxonomy (addressing the Unknown-Skill gap found in Section 16/21).
- Fine-tuning a transformer model on this domain's labelled data to close the gap seen in Section 15.

24. INTERNSHIP LEARNING OUTCOMES

Technical Skills

- Practical, hands-on experience with the full NLP stack: regex, NLTK/spaCy preprocessing, TF-IDF, n-grams, embeddings, NER, Scikit-learn ML pipelines and transformer models.
- Building and maintaining a domain-specific skill taxonomy and a text-normalization engine.

Analytical Skills

- Reading and interpreting Precision/Recall/F1/Accuracy and confusion matrices to compare models objectively rather than by intuition.
- Structured error analysis: categorizing failures instead of treating all errors as one bucket.

Project-Development Skills

- Structuring a multi-stage data pipeline where each stage's output is a clearly defined, reusable artifact (CSV/notebook) for the next stage.
- Turning a data-science pipeline into a business-facing deliverable (dashboard) and a user-facing application (Streamlit/FastAPI).

25. REFERENCES

- Public job-postings dataset (job_id, job_title, company, location, job_description, experience, qualifications, salary range, work type, skills fields).
- Scikit-learn documentation (TfidfVectorizer, Logistic Regression, model evaluation metrics) — <https://scikit-learn.org/>
- Sentence-Transformers documentation — <https://www.sbert.net/>
- Power BI documentation — <https://learn.microsoft.com/power-bi/>